

Dossier técnico — FarmaJyV Venta (POS)

Punto de venta de escritorio para farmacia. Angular 22 + Electron 43 + PrimeNG 22, Firebase Auth y API REST NestJS (`backend-farma-jyv`), compartida con `farma-jyv-admin` .

Documento generado a partir del código en `main` (commit `f2ccc96`). Versión declarada en `environment.version` : `v1.0.0` .

1. Panorama

Dato	Valor
Tipo	App de escritorio (Electron) + web SPA (mismo bundle)
Idioma UI	Español (<code>es</code> default, <code>en</code> presente en <code>public/i18n/en.json</code>)
Autenticación	Firebase Auth (email/password), SDK modular directo
Autorización	<code>GET /auth/me</code> → rol (documento <code>roles</code>) + permisos por área
Persistencia local	<code>localStorage</code> (cola offline, ticket en curso, ventas en pausa)
Impresión	<code>window.print()</code> sobre componentes montados al vuelo; cajón por ESC/POS vía IPC
Pago tarjeta	Mercado Pago Point (órdenes + polling) a través del backend
Tamaño	~8.000 líneas (<code>src</code> + <code>electron</code>), 60 archivos fuente
Pruebas	Vitest, 4 archivos, 34 casos — todos pasan (<code>npx vitest run</code>)

Alcance funcional implementado

Venta (búsqueda/escaneo, ticket, descuentos, pausa), cobro (efectivo, tarjeta Point, mixto), turno de caja (apertura, movimientos, cierre con corte Z propio), cumplimiento COFEPRIS (grupos I–VI, receta, retención, libro de control), desglose fiscal IVA/IEPS, cola offline idempotente, historial del día, reportes locales, anulación (solo `admin`), impresión de ticket/corte/etiqueta, auto-update Electron.

No implementado (ver `GOALS.md`): timbrado CFDI, devoluciones/notas de crédito, lectura X, reportes de servidor, multi-caja con selección al login, pantalla de cliente.

2. Estructura del repositorio

```
electron/          main.js (ventana, IPC, auto-update, cajón), preload.js (contextBridge)
public/i18n/       es.json, en.json
src/app/
  core/            api/ auth/ audio/ firebase/ health/ layout/ notifications/ theme/
  features/pos/    sale/ checkout/ cash-session/ history/ reports/
                  controlled-ledger/ ticket/ services/
  shared/          models/index.ts, utils/{money,tender,taxes,controlled,session-expiry}.ts
src/environments/  environment{,.development,.prod,.electron}.ts
```

Regla de dependencia: `features` → `core` y `shared`; `shared` no importa de `features`; `core` es singleton transversal.

3. Arranque y configuración

app.config.ts

- `provideRouter(routes) + withRouterLocation()` **solo** cuando `environment.isElectron` — Electron carga `index.html` por `file://` y el router de path rompe.
- `provideHttpClient(withInterceptors([authInterceptor]))`.
- `provideTranslateService` con loader HTTP; prefijo `./i18n/` en Electron, `/i18n/` en web.
- `provideFirebase()` — `InjectionToken`s propios (ver 4.1).
- `providePrimeNG` con preset `PosPreset` y `darkModeSelector: false` (caja siempre en claro).

Entornos

Los cuatro archivos comparten proyecto Firebase, `apiUrl` y credenciales de Mercado Pago (`storeId: 79584227`, `posId: 136088410`). Diferencias:

Archivo	production	isElectron
<code>environment.ts</code>	false	false
<code>.development.ts</code>	false	false
<code>.prod.ts</code>	true	false
<code>.electron.ts</code>	true	true

Claves de configuración operativa: `printTicketOnSale` (auto-imprime al cerrar venta), `expiryWarningDays: 30`, `soundsEnabled`, `promos: []` (reglas locales NxM/%), `cashDrawer.printerName` (vacío = cajón deshabilitado), `pharmacy.{name,address,phone,rfc}` (encabezado de tickets).

Nota de seguridad. `environment.ts` versiona la `apiKey` de Firebase (esperado: es pública y se protege por reglas del proyecto) y la licencia de PrimeNG. No hay secretos de servidor en el bundle; el Access Token de Mercado Pago vive en el backend.

Comandos

```
npm start                # ng serve → localhost:4200
npm run build             # producción → dist/farma-jyv-pos/browser
npm run electron:build   # configuración `electron` (baseHref ./, env.electron)
npm run electron:dev     # Electron contra el dev server
npm run electron:start   # build + Electron contra el bundle
npm run electron:dist[:mac|:win] # electron-builder → /release
npm test                 # Vitest
```

Presupuesto de bundle inicial: warning 700 kB, error 1.5 MB.

4. Core

4.1 Firebase sin @angular/fire

`core/firebase/firebase.providers.ts` expone `FIREBASE_APP` y `FIREBASE_AUTH` como `InjectionToken`s creados con `initializeApp` / `getAuth` del SDK modular. Motivo: `@angular/fire` no soportaba Angular 22 al crear el proyecto. Consumo idéntico al de `@angular/fire` salvo el token inyectado. Migración futura: reemplazar `provideFirebase()` por `provideFirebaseApp` / `provideAuth`.

4.2 AuthService

- `user` — signal desde `onAuthStateChanged`.
- `profile` — signal desde `GET /auth/me`, parseado con `parseStaffProfile` (tolera `role` como string o `{slug}`). Deduplicado con `shareReplay(1)` + `finalize` que limpia la petición en vuelo.
- `role`, `roleName`, `permissions`, `isAuthenticated`, `isAdmin` (`role === 'admin'`, exacto porque `assertCanVoidSale` en el backend exige ese slug), `canSell` (`sales:write`).
- `can(area, level)` delega en `hasPermission`, que replica la función homónima del backend **incluido el atajo de admin**: una UI más estricta que la API bloquearía acciones permitidas.
- **Expiración de sesión**: `effect` que, al cambiar el usuario, lee `authTime` del ID token y programa `endExpiredSession()` para el instante que devuelve `getSessionExpiryMs`. El `setTimeout` se acota a 2.147.483.000 ms para no desbordar.

4.3 Guards

- `authGuard` — espera `authStateReady()`, exige `currentUser`; si no, `UrlTree` a `/login`.
- `guestGuard` — en `/login`: si ya hay usuario, resuelve el rol; **sin rol útil hace `signOut` y deja entrar al login** (evita el limbo de un usuario de Firebase sin staff en el backend).

4.4 authInterceptor

1. Pasa de largo cualquier URL que no empiece con `environment.apiUrl`, y también `/health`.
2. Espera `authStateReady()`, adjunta `Authorization: Bearer <idToken>`.
3. Ante 401: notifica con el mensaje real del backend (que distingue "sesión expiró a las 24:00" de "no autorizado"), hace `signOut`, navega a `/login` y re-lanza el error. Flag módulo-global `isLoggingOut` evita

cascadas cuando varias peticiones fallan a la vez.

4. `/auth/me` queda exento del manejo de 401 — de otro modo el propio arranque del perfil dispararía el logout.

4.5 `api.utils.ts`

`unwrapEntity` / `unwrapList(response, key?)` absorben las formas inconsistentes del backend (`data`, `items`, `products`, array plano). `toDate` normaliza `Timestamp` de Firestore, `Date`, epoch, ISO y `{_seconds}`. `getApiErrorMessage` prioriza `error.error.message`. `ApiRequestError` y `getApiErrorStatus` para errores propios.

4.6 `ApiHealthService`

`browserOnline` (eventos `online` / `offline`), `apiOk` (GET `/health` cada 30 s), `degraded = !browserOnline || !apiOk`. El shell pinta banner rojo (sin internet) o ámbar (API caída).

4.7 Otros

- `NotificationService` — wrapper de `MessageService` de PrimeNG; `sessionExpired()` con copy explícito y `life: 10 s`.
- `ScanSoundService` — WebAudio: `ok()` 880 Hz seno, `warn()` 440 Hz triángulo, `error()` 220 Hz cuadrada. Falla en silencio si el navegador bloquea audio sin gesto previo.
- `PosPreset` — preset de tema PrimeNG.

5. Rutas y navegación

```
/login          guestGuard
/              authGuard → Shell
  ''          → redirect /pos
/pos            Sale          (F1)
/pos/historial  SaleHistory   (F3)
/pos/reportes   PosReports
/pos/libro-control ControlledLedger (requiere inventory:read)
**            → /
```

Todo lazy (`loadComponent` / `loadChildren`). `NAV_ITEMS` declara el permiso que resguarda cada pantalla; el shell filtra los enlaces con `AuthService.can()` — un enlace que devolvería 403 no es navegación.

6. Modelo de dominio (`shared/models/index.ts`)

Entidades: `Product` (con `controlledGroup`, banderas fiscales `hasIva` / `hasIvaZero` / `hasIeps` / `iepsRate`), `ProductBatch` (lote + caducidad), `CartLine`, `Sale` / `SaleItem` / `SaleItemTaxes` / `SaleTaxSummary`, `SalePrescription`, `SaleBilling`, `Customer`, `CashSession` / `CashSessionSummary` / `CashSessionCut`, `CashMovement`, `HeldSale`, `StaffProfile` / `RoleSummary` / `RolePermission`.

Campos de `Sale` que importan para cuadrar caja:

- `cashAmount` — parte del total pagada en efectivo (lo que entra al cajón). `null` en ventas previas al split mixto y en tarjeta/transferencia.
- `cardAmount` — parte con tarjeta (monto de la orden Point).
- `amountReceived` / `change` — lo entregado por el cliente y su cambio.
- `taxSummary` — `null` en ventas anteriores al desglose.
- `controlledGroups`, `prescriptionRetained` — rastro COFEPRIS.
- `invoiceStatus: 'pending' | null` — el POS **no** timbra.

`PermissionArea` cubre `dashboard | users | sales | categories | products | suppliers | inventory | invoices | uploads | doctor`; `PermissionLevel` es `read | write`.

7. Reglas replicadas del backend (`shared/utils`)

Cuatro módulos puros que espejean lógica del servidor. **El backend sigue siendo la autoridad**; la copia existe para bloquear en el mostrador y para que el ticket offline sea correcto. Todos tienen spec.

7.1 `money.ts`

Aritmética en centavos enteros: `toCents`, `fromCents`, `roundMoney`, `addMoney`, `subtractMoney`, `compareMoney`, `isMoneyGreaterOrEqual`. Sin esto, el "faltan" de la caja se separa del total que valida la API por punto flotante.

7.2 `tender.ts` — reparto efectivo/tarjeta

`resolveTender({paymentMethod, total, amountReceived, cardAmount})` → `{cashDue, cashAmount, cardAmount, change, shortfall, cashSatisfied, error}`.

Método	cashDue	Notas
cash	total	cambio = recibido – total
card / transfer	0	en card, <code>cardAmount</code> = total
mixed	<code>total – cardAmount</code>	el cambio se calcula contra esa parte, nunca contra el total

Validaciones de `mixed`: `cardAmount` requerido, > 0 , y estrictamente menor que el total (si cubre todo, es venta con tarjeta). Fuente única para checkout, ticket y cola offline.

7.3 `taxes.ts` — desglose con impuestos incluidos

El precio de catálogo es precio al público. Se calcula hacia atrás con el orden legal mexicano: IEPS sobre la base, IVA sobre (`base + IEPS`).

```
gross = base × (1 + iepRate) × (1 + ivaRate)
```

`resolveIvaRate` — `hasIvaZero` gana sobre `hasIva` (medicina de patente 0%); `IVA_RATE = 0.16`. El residuo de redondeo se absorbe en la base para que `base + iep + iva` sea exactamente el importe cobrado (un desglose que no suma se rechaza al timbrar). `prorateDiscount` reparte el descuento de venta

proporcionalmente **antes** de calcular impuestos, con los centavos sobrantes cargados a la partida mayor. `previewTaxSummary` es la secuencia completa que usa el checkout y el ticket offline.

7.4 `controlled.ts` — grupos COFEPRIS (art. 226 LGS)

Grupo	Receta	Folio obligatorio	Se retiene	Libro de control
I estupefacientes	sí	sí	sí	sí
II psicotrópicos	sí	sí	sí	sí
III psicotrópicos menor riesgo	sí	sí	sí	sí
IV antibióticos y otros de receta	sí	no	no	sí
V venta en farmacia sin receta	no	no	no	no
VI venta libre	no	no	no	no

`resolveControlledRequirements(products)` agrega el ticket completo. Si un producto **no** trae `controlledGroup`, cae al flag heredado `requiresPrescription` (deuda de catálogo, ver 13). `validatePrescription` devuelve el primer motivo de bloqueo, con los mismos mensajes del backend. `isValidDoctorLicense` — cédula de 7 u 8 dígitos.

7.5 `session-expiry.ts`

El backend rechaza cualquier token cuyo `auth_time` sea de un día anterior en `America/Mexico_City`: la sesión muere a las **24:00 locales**, no a las 24 h de haber entrado. `getSessionExpiryMs` calcula el inicio del día siguiente por búsqueda binaria en ± 14 h sobre `Intl.DateTimeFormat` — sin offset hardcodedo, sobrevive al horario de verano.

8. Flujo de venta (`features/pos/sale`)

Estado

Signals: `searchTerm`, `results`, `cart`, `heldSales`, `manualDiscounts`, `lastSale`, más los `*Visible` de cada diálogo. Derivados: `subtotal`, `discountTotal`, `total`, `itemCount`, `lastLineId`.

Búsqueda y escaneo

Subject con `debounceTime(500)` + `distinctUntilChanged()` + `switchMap` a `GET /products?search=`. Enter en el buscador:

1. Parsea el prefijo `N*código` (`/^(\d+)\*(.+)$/`), cantidad acotada a 1–999.
2. Busca y toma coincidencia exacta por `sku` o `barcode`; si hay un solo resultado, ese.
3. Sin coincidencia → sonido de error + mensaje distinto para "no encontrado" vs "varios resultados".

Alta al ticket — `addToCart` → `commitAdd`

1. `ensureShiftOpen()`: verifica `sales:write` y turno abierto (si no, abre el diálogo de turno).
2. `stock < 1` → sonido de error + diálogo de **sustitutos** por principio activo.

3. `GET /inventory/batches?productId=` → `sellableBatches` : descarta `quantity <= 0` y vencidos, ordena por caducidad ascendente (**FEFO**).
 - Suma vendible 0 → "El stock disponible está vencido" + sustitutos.
 - Lote más próximo a ≤ `expiryWarningDays` (30) días → sonido de aviso + fecha y días restantes.
 - Si la llamada a lotes falla, se degrada a `product.stock` .
4. Tope por `min(sellableQty, product.stock)` .
5. Producto controlado nuevo en el ticket → `warnIfControlled` : sonido + grupo + si se retiene la receta. Se avisa **al agregar**, no al cobrar.
6. `setCart` pasa siempre por `PromoService.apply` .

Descuentos

`updateLineDiscount` reparte el importe capturado entre promo (calculada) y manual (`manualDiscounts[productId]`), acota al total de línea, y **rechaza > 20 % si el usuario no es admin** .

Autoguardado y pausa

- `CartStorageService` — `effect` que persiste ticket + descuentos manuales en `pos.current-cart.<uid>` en cada cambio; se restaura al entrar (solo si el carrito está vacío) con aviso al cajero. Existe porque la sesión muere a las 24:00 y un 401 cierra sesión: sin esto se reescanea todo.
- `HeldSaleStorageService` — `pos.held-sales.<uid>` ; F6 pausa (etiqueta con hora), botón para retomar. Retomar exige carrito vacío. Clave por `uid` para no mezclar tickets de cajeros en el mismo equipo.

Atajos (host bindings en `document`)

F2 buscar · F4 foco en cantidad de la última línea · F6 pausar · F9 cobrar · Esc limpiar búsqueda, luego vaciar ticket (con `confirm`) · Del / Backspace quitar última línea · + / - / = ajustar cantidad de la última línea.

Getter privado `dialogOpen` cortocircuita **todos** los atajos cuando hay cualquier diálogo abierto. Sin él, Esc cerraba el cobro y vaciaba el ticket. `isTypingTarget` protege los campos de texto de Del y +/- .

9. Cobro (`features/pos/checkout`)

Diálogo de 58 rem en dos columnas: izquierda lo que se teclea (total, método, efectivo/tarjeta), derecha cumplimiento y datos fiscales.

Idempotencia

`idempotencyKey` (uuid v4, con fallback `pos-<base36>-<random>` para WebViews viejos) se genera **al abrir el diálogo** y se reusa en cada reintento de "Confirmar". La referencia enviada a la terminal es `sale-<idempotencyKey>-<intento>` : la orden queda trazable a la venta y el sufijo distingue reintentos, porque una orden muerta sigue ocupando su referencia en Mercado Pago.

Métodos

`cash` · `card` · `mixed` (la UI ofrece tres; el modelo también admite `transfer`). `selectPaymentMethod` protege dos casos: con orden **aprobada** no se cambia de método (dejaría dinero cobrado fuera de la venta; el reembolso debe ser explícito), y con orden viva sin aprobar se cancela antes de cambiar. `mixed` arranca a mitades y mueve el foco al campo que toca teclear.

Tarjeta — Mercado Pago Point

GET /payments/mercadopago/devices?storeId&posId lista terminales. Si `operatingMode === 'STANDALONE'` aparece "Activar PDV" (PATCH .../devices/operating-mode). POST .../orders crea el cobro por `cardChargeAmount` (total, o solo la parte con tarjeta en mixto) y arranca polling `interval(2000)` + `takeWhile(pending, true)` sobre `GET .../orders/:id`.

Estados: `created`, `at_terminal`, `action_required` (pendientes) → `processed` (aprobado), `failed`, `expired`, `canceled`, `refunded`. `cardAmountLocked` congela el monto con tarjeta mientras la orden vive (pendiente o aprobada) y lo libera si falló/expiró, para reintentar con otro reparto sin cerrar el diálogo. `retryCardPayment` no cancela (la orden ya está muerta); `cancelCardPayment` sí hace `DELETE .../orders/:id`.

Bloqueos

`blockers()` lista, **en el orden en que el cajero los resuelve**, por qué no se puede cobrar: receta → reparto inválido → terminal sin aprobar → falta efectivo → RFC/razón social. Se renderiza con `role="alert"` junto al botón. F9 inválido notifica el primero. Regla de diseño: nunca un botón deshabilitado sin motivo visible.

`canConfirm`: `cash` exige `cashSatisfied`; `card` exige orden aprobada; `mixed` exige **ambos** — con uno solo la venta quedaría cobrada a medias.

Cumplimiento y fiscal

Panel COFEPRIS lista qué partida es controlada y su grupo; campos de receta (médico, cédula, folio) con obligatoriedad dinámica; casilla de retención en I-III. Cliente y facturación **plegados** por defecto (la venta típica es público general). `usoCfdi`: G03, D01, S01. Facturar solo marca `invoiceStatus`: `pending` — el POS no timbra. Desglose base/IEPS/IVA/total visible antes de cobrar.

Teclado y foco

F9 cobra desde cualquier campo (escuchado en `document` porque el diálogo de PrimeNG monta fuera del componente). Enter sobre el monto también cobra. Alt+1/2/3 cambia método — con Alt para no comerse los dígitos. Al abrir, el foco va al campo que bloquea: receta si el grupo la exige, efectivo si no. `focusInput` resuelve el `input` real tanto de `pInputText` como del wrapper de `p-inputnumber`.

Confirmación

`SaleService.buildPayload(...)` → POST /sales. `prescriptionRetained` se envía **solo** cuando el grupo lo exige. Los datos de receta se envían aunque el grupo no la pida (V/VI) si el cajero los capturó: trazabilidad ya escrita a mano. Éxito con efectivo o mixto → `CashDrawerService.open()`.

10. Cola offline

`SaleService` con `localStorage['pos.pending-sales']`.

- **Encolado**: solo ante `HttpErrorResponse.status === 0` (sin red). Devuelve una `Sale` sintética con `id: offline-<ts>`, `folio: PENDIENTE-<ts>`, `taxSummary` calculado localmente y `change / cashAmount / cardAmount` resueltos por `resolveTender`, para que el ticket se imprima igual.
- **Flush**: en el evento `online` y en reintentos manuales. `concatMap` — **en serie**, para no perder el orden de folios ni abrir N peticiones al recuperar la red. Cada venta viaja con su `idempotencyKey` original: si el POST

anterior sí llegó y se perdió la respuesta, el backend devuelve la venta ya registrada en vez de duplicarla. Flag `flushing` evita solapes.

- **Bloqueo:** un 4xx (excepto 401, 408, 429) marca `blockedReason` y la venta deja de reintentarse — turno cerrado, sin stock, orden Point reusada no se arreglan reintentando. El cajero las revisa desde el badge rojo de la barra y decide **reintentar** (misma llave) o **descartar** (con `confirm`; descartar en silencio sería perder una venta ya cobrada).

Badge ámbar = pendientes de sync. Badge rojo = rechazadas.

11. Turno de caja

`CashSessionService` sobre `/cash-sessions: current`, `POST` / (abrir con `openingAmount`), `GET :id/summary`, `POST :id/close` (`countedCashAmount`), `GET/POST :id/movements`.

`CashSessionDialog` tiene tres modos derivados: `open` (sin sesión; no cerrable con Esc ni clic afuera — sin turno no se vende), `close` (resumen + captura del efectivo contado + diferencia en vivo), `result` (turno cerrado, imprimir corte). El efectivo esperado sale de `expectedCashAmount` del backend con fallback a `summary.cashInDrawer`; el contado se precarga con el esperado.

Movimientos de caja (`deposit` / `withdrawal` / `expense`, con monto y motivo obligatorios) solo se ofrecen a `admin` — el botón y el propio componente están gateados en la plantilla.

12. Pantallas de consulta

Historial (`/pos/historial`) — ventas del turno actual, o del día si no hay turno; `includeVoided: true`. Filtro local por folio o nombre de producto. Detalle en diálogo con el ticket embebido, reimpresión, y anulación (`POST /sales/:id/void`) solo para `admin` y con `confirm`.

Reportes (`/pos/reportes`) — alcance turno o día, calculados **en el cliente** sobre `listAll` (paginación recursiva de 100 en 100). Métricas: venta neta, ticket promedio, descuentos, anuladas, por método, por hora (con barra proporcional), top 10 por cantidad e importe. `cashCollected` suma con la misma regla del corte del backend: `cashAmount`, con fallback `recibido - cambio` para ventas viejas — sumar el total de una venta mixta contaría también lo de la tarjeta. Imprimible con `window.print()`.

Libro de control (`/pos/libro-control`) — `GET /inventory/controlled-ledger`, solo lectura (los renglones los escribe el backend dentro de la transacción de venta/anulación/devolución). Filtros por rango, rangos rápidos (hoy / 7 / 30 días) y grupo; `to` se envía como `<fecha>T23:59:59.999` porque el backend filtra por instante. Renglones con folio, movimiento, producto, grupo, cantidad **con signo**, lotes, receta y paciente. Resumen por grupo y piezas netas. Gateado por `inventory:read`, mismo permiso del endpoint. Se consulta desde la caja porque la visita de verificación ocurre en el mostrador.

13. Impresión

`TicketPrintService.printHost` crea el componente con `createComponent`, lo adjunta al `ApplicationRef`, fuerza `detectChanges`, llama `window.print()` y limpia en `afterprint` o a los 1500 ms (idempotente por flag

cleaned).

Tres plantillas: `SaleTicket` (encabezado de farmacia, partidas, reparto efectivo/tarjeta cuando aplica, desglose fiscal, folio de receta y retención, grupos COFEPRIS, leyenda de impuestos incluidos y nota del art. 226 LGS), `CashCutTicket` (corte Z con desglose por método y movimientos), `ProductLabel` .

`CashDrawerService` → IPC `open-cash-drawer` → `electron/main.js` escribe el pulso ESC/POS `1B 70 00 19 FA` a un archivo temporal y lo manda a la impresora: `copy /b` en Windows, `lp -d <cola> -o raw` en el resto. No-op si no es Electron o si `printerName` está vacío.

14. Electron

`main.js` — ventana única 1360×900 (mín. 1024×700), `contextIsolation: true` , `nodeIntegration: false` , `webSecurity: true` . Carga el dev server si `NODE_ENV=development` o `--dev` ; si no, `dist/farma-jyv-pos/browser/index.html` , y si falta el build sale con mensaje explícito. Locale forzado a `es-MX` . `setPermissionRequestHandler` solo concede `geolocation` .

IPC expuesto por `preload.js` vía `contextBridge` como `window.electronAPI` : `getAppVersion` , `getDeviceInfo` (MAC + hostname → `id` estable para identificar el equipo de venta), `openCashDrawer` .

Auto-update con `electron-updater` (`autoDownload = true` , instala al cerrar), solo fuera de dev, con feed sobrescribible por `UPDATE_URL` . Todos los errores del updater son no-op logueados — un feed caído no debe tumbar la caja.

`electron-builder` (clave `build` de `package.json`): `appId mx.farmajyv.pos` , salida `/release` , asar. macOS `dmg+zip arm64/x64` (`identity: null` , **sin firmar**), Windows `nsis+zip`. Feed de publicación `https://updates.farmajyv.mx/pos` — hoy placeholder.

15. Superficie de API consumida

Método	Ruta	Uso
GET	/auth/me	perfil, rol y permisos
GET	/health	banner de degradación
GET	/products?search=	búsqueda y escaneo
GET	/inventory/batches?productId=	lotes, FEFO, caducidad
GET	/inventory/controlled-ledger	libro de control
GET/POST	/customers	búsqueda y alta rápida
GET	/sales	historial y reportes (paginado)
GET	/sales/:id	detalle
POST	/sales	registrar venta (idempotente)
POST	/sales/:id/void	anular (solo admin)
GET	/cash-sessions/current	turno vigente
POST	/cash-sessions	abrir turno
GET	/cash-sessions/:id/summary	resumen previo al corte
POST	/cash-sessions/:id/close	cerrar turno
GET/POST	/cash-sessions/:id/movements	entradas y salidas de efectivo
GET	/payments/mercadopago/devices	terminales Point
PATCH	/payments/mercadopago/devices/operating-mode	activar PDV
POST	/payments/mercadopago/orders	enviar cobro a la terminal
GET	/payments/mercadopago/orders/:id	polling del cobro
DELETE	/payments/mercadopago/orders/:id	cancelar cobro

Endpoints ya disponibles en el backend y **no** consumidos aún: recibos (/sales/:id/receipt , /sale-returns/:id/receipt), devoluciones (/sale-returns), reembolso Point, lectura X (/cash-sessions/:id/x-report), reportes de servidor (/reports/*), alertas de inventario (/inventory/alerts), auditoría (/audit-logs).

16. Convenciones de código

Standalone por defecto (sin standalone: true), ChangeDetectionStrategy.OnPush en todo componente, signals para estado y computed para derivados (nunca mutación directa), input() / output() en lugar de decoradores, objeto host en lugar de @HostBinding / @HostListener , control de flujo nativo @if / @for / @switch , bindings class / style en lugar de ngClass / ngStyle , inject() sobre constructor, servicios providedIn: 'root' , plantillas y estilos externos junto al .ts .

@ngx-translate v18: TranslatePipe / TranslateDirective standalone, **no** TranslateModule . PrimeNG p-table : #header / #body , no pTemplate .

17. Pruebas

`npx vitest run` → 4 archivos, 34 casos, todos verdes (296 ms). Cubren exactamente las reglas replicadas del backend: `tender.spec.ts` , `taxes.spec.ts` , `controlled.spec.ts` , `session-expiry.spec.ts` .

Sin cobertura de pruebas: componentes (`Sale` , `Checkout` , diálogos), servicios HTTP, cola offline, `PromoService` , e2e. La cola offline y el pago mixto son las dos áreas con mayor riesgo y ninguna tiene test automatizado — hoy dependen de verificación manual (ver `GOALS.md` , "Por verificar").

18. Riesgos y deuda técnica

Operativos / de datos

1. **Catálogo sin `controlledGroup`** : el POS cae al flag heredado `requiresPrescription` , que no exige folio ni retención ni deja renglón en el libro. Es la brecha de cumplimiento más directa y se cierra capturando el grupo en el catálogo, no en el POS.
2. **Point sin prueba física:** código completo, emparejado de TPV y venta real de prueba pendientes.
3. **Modo offline sin prueba real sin red:** flush, conflictos y mensajes están escritos pero no ejercitados end-to-end.
4. **Reportes calculados en el cliente** sobre `listAll` paginado: con volumen alto, muchas peticiones y agregación en memoria. La API ya expone `/reports/*` .
5. **Corte Z reimpresso localmente:** `POST :id/close` ya devuelve el recibo renderizado (58/80 mm) y el POS lo descarta.
6. **Sin lectura X:** solo hay corte Z al cerrar; el turno no puede fotografiarse sin cerrarlo.
7. **Sin devoluciones:** `POST /sale-returns` existe en el backend; en caja solo hay anulación total de la venta.
8. **CFDI:** se captura `billing` y se marca `pending` ; sin PAC elegido no hay timbrado. No prometer facturación en caja.

Técnicos

9. `window.confirm` / `window.alert` nativos en varios flujos (vaciar ticket, quitar línea > 5 piezas, anular, descartar venta bloqueada) — en Electron son diálogos del sistema, inconsistentes con el resto de la UI.
 10. `isLoggingOut` es estado global de módulo en el interceptor: correcto en la app real (una ventana), pero frágil en pruebas.
 11. Firma de código ausente en macOS (`identity: null`) y feed de actualizaciones apuntando a un host placeholder.
 12. `en.json` existe y está completo pero no hay selector de idioma en la UI.
 13. `PromoService` lee reglas de `environment.promos` — cambiar una promo exige recompilar y redistribuir el instalador.
 14. Multi-caja: `storeId` / `posId` son fijos en el entorno; no hay selección de caja al iniciar sesión.
-

19. Referencias

- `CLAUDE.md` — guía de arquitectura para agentes.
- `GOALS.md` — backlog priorizado (P0–P3) y bitácora de lo hecho.
- `README.md` — comandos y stack.
- `graphify-out/` — grafo de conocimiento del código (`graphify query` , `path` , `explain`).